

E COMMERCE SYSTEM

BY

Name: Bagmita Pokhrel

T.U. Registration No.: 7-2-1125-30-2022

Exam Roll No: 15251/22

&

Name: Garima Rana Magar

T.U. Registration No.: 7-2-1125-38-2022

Exam Roll No: 15260/22

Ambikeshwari Campus

A Project Report Submitted to

Faculty of Management, Tribhuvan University

in partial fulfillment of the requirements for the degree of

Bachelor of Information Management (BIM)

Ghorahi, Dang

January/2026

STUDENT DECLARATION

This is to certify that We have completed the Project entitled “**E Commerce System**” under the guidance of “**Mr. Rajendra Kandel**” in partial fulfillment of the requirements for the degree of **Bachelor of Information Management** at Faculty of Management, Tribhuvan University. This is our original work, and We have not submitted it earlier elsewhere.

Date: January 25, 2026

Signature:

Bagmita Pokhrel

Signature:

Garima Rana Magar

CERTIFICATE FROM THE SUPERVISOR

This is to certify that the project entitled “**E Commerce System**” is an academic work done by “**Ms. Bagmita Pokhrel**” and “**Ms. Garima Rana Magar**” submitted in the partial fulfillment of the requirements for the degree of Bachelor of Information Management at Faculty of Management, Tribhuvan University under my guidance and supervision. To the best of my knowledge, the information presented by them in the project report has not been submitted earlier.

Signature of the Supervisor

Lecturer Rajendra Kandel

Date: August 20, 2025

APPROVAL SHEET

This is to certify that the project titled “**E Commerce System**” submitted by “**Ms. Bagmita Pokhrel**” and “**Ms. Garima Rana Magar**” has been examined and approved. In our opinion, it meets the required scope and quality standards for a project submitted in partial fulfillment of the requirements for the degree of Bachelor of Information Management (BIM).

Approval Panel:

S.N.	Name	Designation	Signature
1	Mr. Rajendra Kandel	Project Supervisor	
2	Mr. Narendra Banjade	Program Coordinator	
3	Mr. Sandesh Poudel	Internal Examiner	
4	Mr. Harendra Subedi	External Examiner	

Date of Defense: January 26, 2026

Department: Bachelor of Information Management

Faculty: Faculty of Management

ACKNOWLEDGEMENT

This project report has been prepared in partial fulfillment of the requirements for the degree of **Bachelor of Information Management (BIM)** as prescribed by the **Faculty of Management, Tribhuvan University**. This is a team project assigned during the sixth semester, and the work has been completed jointly by both project partners.

We would like to express our sincere gratitude to our project supervisor, **Mr. Rajendra Kandel**, for his valuable guidance, constructive feedback, and continuous support throughout the development of this project. His supervision and expertise played a vital role in the successful completion of this work.

Furthermore, we are thankful to the faculty members and staff of **Ambikeshwari Campus** for providing the necessary academic environment and support during the project period. Finally, we would like to thank our family and friends for their constant encouragement.

Bagmita Pokhrel (15251/22)

Garima Rana Magar (15260/22)

ABSTRACT

The rapid growth of internet penetration and digital payment ecosystems in Nepal has significantly increased the demand for reliable and user-friendly e-commerce platforms. This project, titled E Commerce System, was designed and developed to provide an efficient digital storefront tailored to the Nepali market. The primary objective of the system was to enable users to register, browse products by category, manage a shopping cart, and place orders seamlessly. To accommodate local consumer preferences, the system integrated online payment gateways along with a Cash on Delivery (CoD) option. Additionally, a comprehensive administrative module was implemented to support inventory management, category control, user account administration, and order processing. The system was developed using the Django web framework, with HTML, CSS, Bootstrap, and JavaScript employed for the frontend, while SQLite was used for database management. An Agile development methodology was adopted to support iterative development, flexibility, and timely delivery of system components. The successful implementation of this project demonstrated a modern, modular, and extensible e-commerce solution capable of supporting small-scale digital commerce within Nepal's rapidly expanding e-commerce landscape.

Keywords: *E-commerce, Django Framework, Online Ordering System, Digital Payment, Agile Methodology.*

TABLE OF CONTENTS

STUDENT DECLARATION	ii
CERTIFICATE FROM THE SUPERVISOR	iii
APPROVAL SHEET.....	iv
ACKNOWLEDGEMENT	v
ABSTRACT.....	vi
TABLE OF CONTENTS	vii
LIST OF FIGURES	ix
LIST OF TABLES	x
LIST OF ABBREVIATIONS	xi
Chapter 1: Introduction	1
1.1 Background of the Project	1
1.2 Problem Statement	1
1.3 Objectives	1
1.4 Review of Related Work and Literature	2
1.5 Development Methodology	2
1.5.1 Requirement Gathering.....	3
1.5.2 Analysis.....	3
1.5.3 Design	3
1.5.4 Implementation	3
1.5.5 Testing.....	4
1.5.6 Maintenance and Feedback.....	4
1.6 Scope and Limitations.....	4
1.6.1 Scope.....	4
1.6.2 Limitations	5
1.7 Report Organizations	5
Chapter 2: System Development Process	6
2.1 Requirement Analysis	6
2.1.1 Functional Requirements	6
2.1.2 Non-Functional Requirements	7
2.1.3 Feasibility Study	8
2.2 Structured Oriented Modelling	9

2.2.1 Data Flow Diagram (DFD)	9
2.3 System Design	11
2.3.1 Activity Diagram.....	11
2.3.2 Sequence Diagram	13
2.3.3 Entity-Relationship Diagram (ERD).....	14
2.4 System Implementation	17
2.4.1 Tools and Technologies Used.....	17
2.4.2 Module Description	18
2.4.3 Testing.....	25
Chapter 3: Conclusion and Recommendation.....	28
3.1 Summary.....	28
3.2 Conclusion	29
3.3 Recommendation	29
References.....	31
Appendices.....	32

LIST OF FIGURES

Figure 1.1: Agile Development Cycle	2
Figure 2.1: Use Case Diagram	6
Figure 2.2: Class Diagram	7
Figure 2.3: Gantt Chart	9
Figure 2.4: Data Flow Diagram	10
Figure 2.5: Activity Diagram	13
Figure 2.7: ER Diagram	17
Figure 2.8: Registration Form.....	19
Figure 2.9: Login Form.....	19
Figure 2.10: User Profile.....	20
Figure 2.11: Home Page	20
Figure 2.12: Product Listing	21
Figure 2.13: My Cart.....	21
Figure 2.14: Checkout Form.....	22
Figure 2.15: Choose Payment Method.....	22
Figure 2.16: Esewa Login Page	23
Figure 2.17: Successful Payment.....	23
Figure 2.18: Admin Dashboard.....	24
Figure 2.19: Add Product.....	24

LIST OF TABLES

Table 2.1: Gantt Chart.....	9
Table 2.2: User Entity	15
Table 2.3: Product Entity	15
Table 2.4: Category Entity	15
Table 2.5: Order Entity	15
Table 2.6: OrderItem Entity	16
Table 2.7: Test Case 1	25
Table 2.8: Test Case 2.....	25
Table 2.9: Test Case 3.....	26
Table 2.10: Test Case 4.....	26
Table 2.11: Test Case 5.....	26
Table 2.12: Test Case 6.....	27
Table 2.13: Test Case 7.....	27
Table 2.14: Test Case 8.....	27

LIST OF ABBREVIATIONS

API	Application Programming Interface
CoD	Cash on Delivery
CRUD	Create, Read, Update, and Delete
CSS	Cascading Style Sheets
DBMS	Database Management System
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JS	JavaScript
MVC	Model-View-Controller
MVT	Model-View-Template
SDLC	Software Development Life Cycle
SQL	Structured Query Language
UAT	User Acceptance Testing
UI	User Interface
URL	Uniform Resource Locator

Chapter 1: Introduction

1.1 Background of the Project

An E-Commerce System is a digital platform that allows businesses to sell products and services over the internet. In the context of Nepal, the retail landscape is rapidly shifting from traditional storefronts to digital marketplaces. E Commerce System (Luniva) is a web-based e-commerce platform designed to bridge this gap for local small and medium-sized enterprises (SMEs). Developed using the Django framework and a persistent SQLite database, the system provides a structured environment for product browsing, secure checkout, and automated record-keeping. By integrating local payment gateways like eSewa, the system moves businesses away from manual ledger-based operations to an automated, efficient digital storefront.

1.2 Problem Statement

In Nepal, many small and medium-sized businesses still rely on manual methods or social media platforms like Facebook and Instagram to take orders and manage payments. This informal "chat-to-order" approach often leads to several critical issues:

- **Operational Errors:** Manual entry of orders often results in double-ordering or shipping the wrong items.
- **Payment Insecurity:** Relying on manual verification of payment screenshots (eSewa/Bank) is time-consuming and prone to fraud.
- **Poor Record-Keeping:** Without a centralized database, tracking sales history and inventory levels becomes nearly impossible as the business scales.
- **Customer Dissatisfaction:** Lack of transparent pricing and real-time stock availability often frustrates potential buyers.

Therefore, there is a growing need for a dedicated digital solution like Luniva that can simplify ordering and payment processes while providing a professional interface for local consumers.

1.3 Objectives

The primary objective of this project is to develop a functional and secure e-commerce application. The specific objectives are as follows:

- To digitalize the manual ordering and payment process for local vendors.
- To design a responsive frontend using HTML, CSS, and Bootstrap for better accessibility.
- To implement dynamic user interactions, such as real-time cart updates, using JavaScript.

- To manage product, user, and order data efficiently using a SQLite database.
- To provide a secure admin dashboard for managing inventory and verifying digital payments.

1.4 Review of Related Work and Literature

Recent studies in the Nepali IT sector indicate that while large-scale platforms like Daraz dominate the market, they are often too complex or expensive for local SMEs due to high commission rates. Most existing local research focuses on PHP-based solutions. This project explores the use of Python Django with, which offers higher security "out of the box."

Literature on local digital payments highlights that eSewa remains the most trusted gateway for Nepali users. By integrating this, Luniva E-commerce System addresses the "Trust Gap" identified in previous studies of Nepali e-commerce. Furthermore, using JavaScript for client-side validation ensures that the system is optimized for the low-bandwidth internet often found in rural parts of Nepal.

1.5 Development Methodology

This project utilizes the Agile Development Methodology, specifically an iterative approach that allows for continuous refinement. Unlike the traditional Waterfall model, Agile enables the development of Luniva E-commerce System in small, functional increments (Sprints). This is particularly effective for e-commerce, where UI feedback and payment gateway testing require frequent adjustments.

The development process was divided into the following phases:

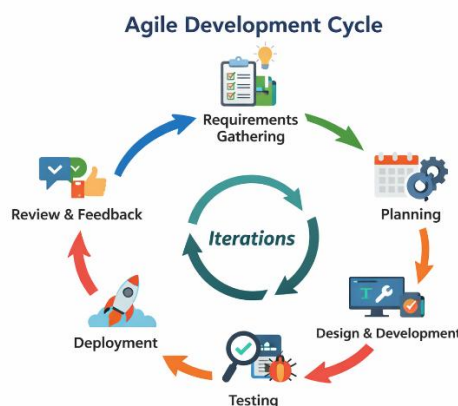


Figure 1.1: Agile Development Cycle

1.5.1 Requirement Gathering

- Discussions were conducted with small business owners and potential users to identify their specific requirements.
- Existing e-commerce platforms were analyzed to identify essential features such as product listing, shopping cart functionality, and online payment mechanisms.
- Both functional and non-functional requirements for user and administrative modules were gathered and documented.
- Features were prioritized and implemented through short, iterative development cycles (sprints).

1.5.2 Analysis

Requirements were analyzed and documented in user stories, focusing on practical e-commerce scenarios.

Example Requirements:

- Admin can manage products, categories, and orders.
- Users can register, browse products, place orders, and make payments online.
- Secure storage of user and order data using Django's built-in security features

1.5.3 Design

System architecture and database schema were designed incrementally as we added new features.

We created UML diagrams (Use Case, Activity) and flowcharts to visualize system processes like the checkout flow.

UI wireframes were designed for iterative review and feedback to ensure usability.

Database tables in SQLite include users, products, categories, orders, and payments.

1.5.4 Implementation

Technologies Used: Python (Django backend), SQLite (database), and HTML/CSS/JavaScript (frontend).

We developed the project in Sprints, focusing on modules incrementally:

- **User Module:** registration, product browsing, cart, checkout, and payment.
- **Admin Module:** product, category, order, and user management.
- **Payment Integration:** Secure connection with the eSewa API.

We implemented reusable functions for database operations and order processing to keep our code clean.

1.5.5 Testing

Continuous Testing: We tested throughout each sprint to identify and fix bugs immediately.

Functional Testing: Verified login, product browsing, order placement, and payment processing.

Validation Testing: Ensured correct data entry and form validations (e.g., email format, required fields).

Security Testing: Restricted admin access and ensured user data was handled safely.

Usability Testing: Collected feedback from peers and supervisors to improve the interface and navigation.

1.5.6 Maintenance and Feedback

- Bugs and issues identified during our testing sprints were promptly fixed.
- Iterative feedback allowed for UI improvements, smoother navigation, and better error handling.
- We ensured the system remained flexible, reliable, and user-friendly throughout development.

1.6 Scope and Limitations

1.6.1 Scope

The Luniva platform is designed to provide a modern e-commerce solution for online ordering and payment. As partners, we defined the scope to include:

- **User Experience:** Allowing users to register, search for specific products, and place orders online efficiently.
- **Digital Payments:** Enabling secure online transactions through the integration of the eSewa payment gateway.
- **Management Dashboard:** Providing an admin interface for us to manage products, categories, and track customer orders in real-time.
- **Responsive Design:** Offering a user-friendly interface accessible on both desktop and mobile devices using Bootstrap.
- **Order Tracking:** Supporting order management for both users (order history) and administrators (status updates).

The project focuses on simplifying the online shopping experience for customers while giving small businesses a structured, digital platform to manage their products.

1.6.2 Limitations

Despite its robust features, we identified several limitations in the current version of Luniva:

- **Local Hosting:** The system is currently hosted in a local development environment and has not been deployed to a live cloud server.
- **Advanced Features:** High-level features like AI-based product recommendations or real-time sales analytics are not included.
- **Payment Restrictions:** The system is limited to the Nepalese payment ecosystem, primarily eSewa, with no support for international gateways or credit cards.
- **Database Scaling:** We used SQLite for its simplicity, which may not be optimized for extremely large product catalogs or high concurrent traffic.
- **Support Tools:** Automated customer support features, such as live chatbots or ticketing systems, are not part of this version.

1.7 Report Organizations

This report is organized into three main chapters, documenting our journey from planning to the final product:

- Chapter I – Introduction: Covers the background, problem statement, objectives, the Agile methodology we followed, the project scope, and its limitations.
- Chapter II - System Development Process: This is the technical core, describing our requirement analysis, system design (DFDs and ERDs), implementation details, and the results of our testing phases.
- Chapter III - Conclusion and Recommendations: Summarizes our project results, presents our conclusions, and suggests future improvements for Luniva.

Chapter 2: System Development Process

2.1 Requirement Analysis

Requirement analysis is the process of defining the system's requirements, i.e., what the system should do and how it should do it. In the case of Luniva E-commerce System, the requirement analysis is centered on developing an e-commerce website that is modern, user-friendly, and facilitates online orders and payments. Requirements were collected through discussion, observation of existing systems, and considering the needs of the customers and admin.

2.1.1 Functional Requirements

Users should be able to register, login, and manage their accounts.

- Users should be able to view products, search for products, and add products to the cart.
- Users should be able to place orders and make payments.
- Admin should be able to manage products, categories, users, and orders.
- Order history, payments, and delivery status should be tracked.

Use Case Diagram

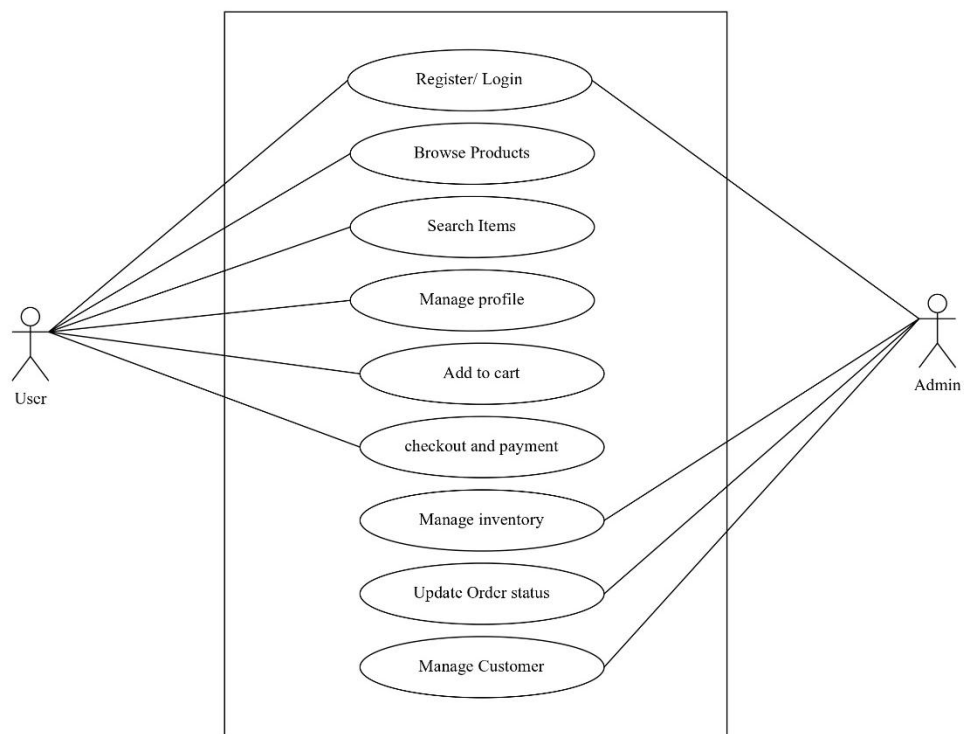


Figure 2.1: Use Case Diagram

Class Diagram

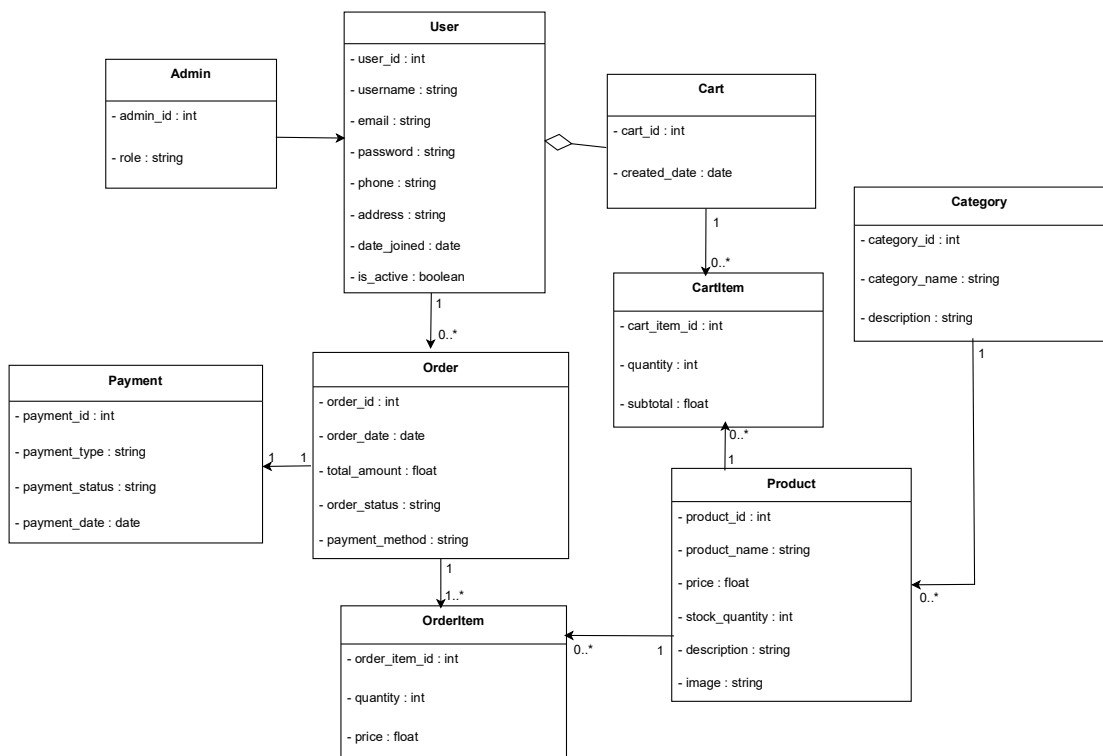


Figure 2.2: Class Diagram

2.1.2 Non-Functional Requirements

Non-functional requirements focus on how well the system performs its functions. For Luniva, these requirements ensure the website is secure, efficient, reliable, and user-friendly, directly reflecting your implementation.

i. Performance:

The website is optimized to load home pages, product pages, and search results within 2–3 seconds.

The system efficiently handles multiple concurrent users, allowing smooth browsing and shopping even when several customers are online.

ii. Security:

User passwords are stored in encrypted format using Django’s built-in security features.

Admin-only functions (like managing products, categories, and orders) are restricted with proper authentication.

Online payments via eSewa are securely integrated, ensuring that sensitive payment data is not stored on the server.

iii. Usability:

The interface is simple, intuitive, and responsive, accessible on desktop, tablet, and mobile devices.

Navigation menus, search bar, product categories, cart, and checkout pages are designed to minimize clicks and user errors.

Form validations ensure users enter correct data during registration, login, and checkout.

iv. **Reliability:**

The system has been tested for bugs and errors, ensuring users experience minimal downtime.

Database transactions are handled carefully to avoid order duplication or data loss.

v. **Scalability:**

Luniva's database and code structure allow future expansion to support more products, users, and even multiple vendors.

Additional payment gateways, categories, or promotional features can be added without major system redesign.

vi. **Maintainability:**

Modular coding and clear separation of frontend and backend make the system easy to update.

Admin functions allow quick changes to products, categories, and user details without developer intervention.

2.1.3 Feasibility Study

Before starting development, a feasibility study was conducted to ensure that Luniva could be implemented successfully.

- i. **Technical Feasibility:** The project uses Python (Django) for the backend, SQLite for the database, and HTML/CSS/JavaScript for the frontend, which are widely supported and compatible technologies. Since SQLite is a serverless database integrated directly with Django, it ensures high portability. Tools like VS Code, Bootstrap, and XAMPP environment ensure efficient development and local server management.
- ii. **Operational Feasibility:** Luniva provides a user-friendly interface for both customers and administrators. Training requirements for shopkeepers are minimal, as the dashboard is intuitive and straightforward. The system effectively automates manual tasks like order tracking and payment verification through eSewa, making it operationally viable for the local market.
- iii. **Schedule Feasibility:** The project is planned for completion within the BIM sixth semester timeframe. By following the Agile approach, the development, testing, and report preparation phases were clearly defined into sprints to ensure timely submission before the final defense.

Table 2.1: Gantt Chart

S.N.	Phase	Start Time	End Time
1	Requirement Gathering	20th Aug	25th Aug
2	Planning	26th Aug	1st Sep
3	Design and Development	2nd Sep	5th Oct
4	Implementation	6th Oct	13th Oct
5	Testing and QA	14th Oct	20th Oct
6	Review and Demo	21st Oct	27th Oct
7	Retrospective	28th Oct	5th Nov

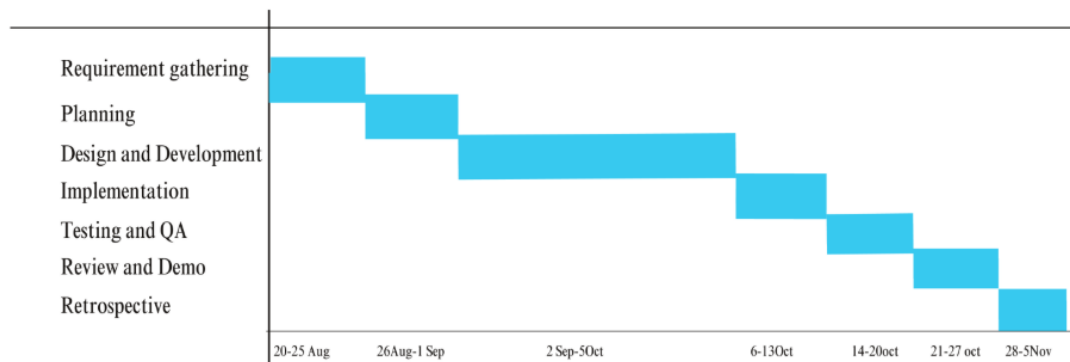


Figure 2.3: Gantt Chart

2.2 Structured Oriented Modelling

Structured-oriented modelling is a method used to design a system logically and visually before actual development. It focuses on the flow of data and how different processes interact within the application. This approach helps in organizing the Luniva E-commerce System in a clear, modular, and hierarchical way. For this project, structured modelling includes the following components:

2.2.1 Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) is a graphical representation that shows how data moves through the system. It represents the transformation of data from input to output and is commonly used during the system analysis phase to define requirements. Based on the system's architecture, the primary components are:

- i. **Process:** A process transforms input data into output within the system. In the Luniva system, key processes include "Product Management," "Cart Processing," "Order Placement & Billing," and "Payment Verification." These are represented by ovals (or rounded rectangles) labeled with descriptive names.

- ii. **Data Flow:** Data flow shows the movement of information between different components. Examples include "Product Search" queries moving from the Customer to the system, or "Payment Confirmation" status moving from the eSewa API back to the system to finalize an order. It is represented by arrows labeled with the specific data type.
- iii. **Data Store:** Represents the storage locations for data used later in the system, such as database tables:
 - **D1. Users DB:** Stores customer and admin credentials, contact details, and roles.
 - **D2. Products DB:** Stores product details, categories, prices, stock, and images.
 - **D3. Orders DB:** Stores order details including products, quantities, user info, payment status, and timestamps.
- iv. **External Entity:** These are individuals or external systems that interact with Luniva but lie outside its core boundaries. The primary external entities are the Customer, the Admin, and the eSewa Payment Gateway.

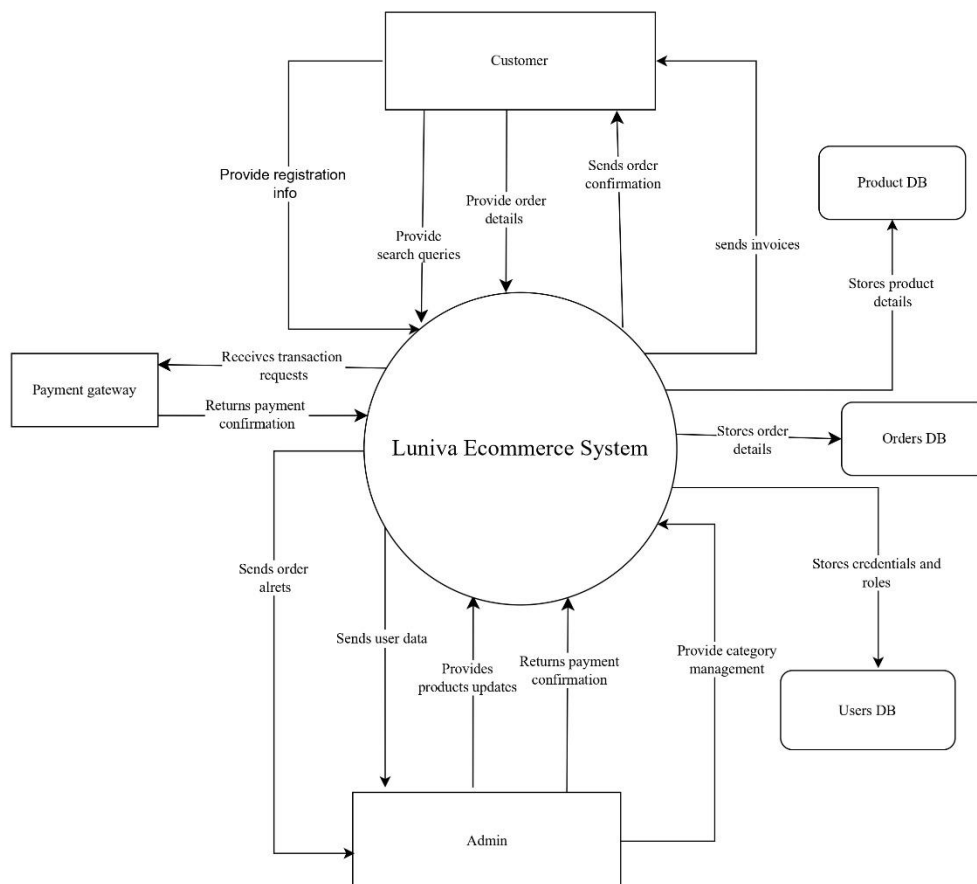


Figure 2.4: Data Flow Diagram

2.3 System Design

System design is the planning and organization of the structure and components of the system before the development process is initiated. It focuses on how the system will fulfill the requirements identified during the analysis phase. In the case of Luniva, the design includes the planning and organization of the functionality of the system, including the flow of data, user interaction, and database organization. This phase ensures that the system is scalable, secure, and correctly incorporates the business logic of an e-commerce platform.

- **Core Modules:** The system is organized into distinct modules to ensure a clean architecture. These include Product Management (for inventory control), Cart and Orders (for purchase logic), Payment Module (handling eSewa and CoD), and User Management (for authentication and profiles).
- **User Interfaces:** The system features separate interfaces tailored for two types of actors: Customers (for browsing and purchasing) and Administrators (for managing the store backend).
- **Database Planning:** This involves the structural organization of the SQLite database. Planning ensures that relational data is stored efficiently across tables for categories, products, orders, users, and payment transactions, maintaining data integrity through primary and foreign keys.
- **Tools and Technologies:** The system utilizes Django templates, HTML, CSS, and Bootstrap for a responsive frontend. The backend logic is powered by Django and Python, with SQLite as the serverless database management system. Additionally, JavaScript is heavily utilized for dynamic UI updates—such as adding products to the cart and updating quantities without a page reload while the eSewa API is integrated for secure digital transactions.

2.3.1 Activity Diagram

The Activity Diagram is a behavioral UML diagram that illustrates the flow of control and data within the Luniva E-commerce System. It maps the step-by-step journey of a user from the initial landing page to the final payment and order confirmation.

- **Logic Visualization:** It provides a clear visual map of decision points, such as checking if a user is authenticated via Django's session management and verifying if a product is in stock before allowing it to be added to the cart.
- **JavaScript Interactivity:** The diagram highlights the role of JavaScript in the "Manage Cart" phase, where the system updates the cart quantities and totals dynamically on the client side before sending the data to the server.
- **Workflow Optimization:** It ensures a smooth transition from product selection to the checkout process, specifically handling the logic branch where a user

chooses between eSewa (External API redirect) and Cash on Delivery (Internal database update).

- **Validation Tool:** It serves as a guide for implementing the "if-then" logic in the Django backend, ensuring that the system only processes orders when the payment gateway returns a "Success" status or when the user confirms a manual delivery.

Key Activities in Luniva:

- **Search/Browse:** User interacts with the UI to find products.
- **Add to Cart (JS):** JavaScript intercepts the click, updates the SQLite database or session, and reflects the change on the UI instantly.
- **Checkout:** The system checks user login status; if not logged in, it redirects to the Login/Register activity.
- **Payment Selection:** The user makes a decision between Digital Payment (eSewa) or Offline Payment (CoD).
- **Payment Processing:** If eSewa is chosen, the system redirects to the eSewa portal, waits for a response, and then updates the order status in SQLite.
- **Confirmation:** A final order summary is generated for the user.

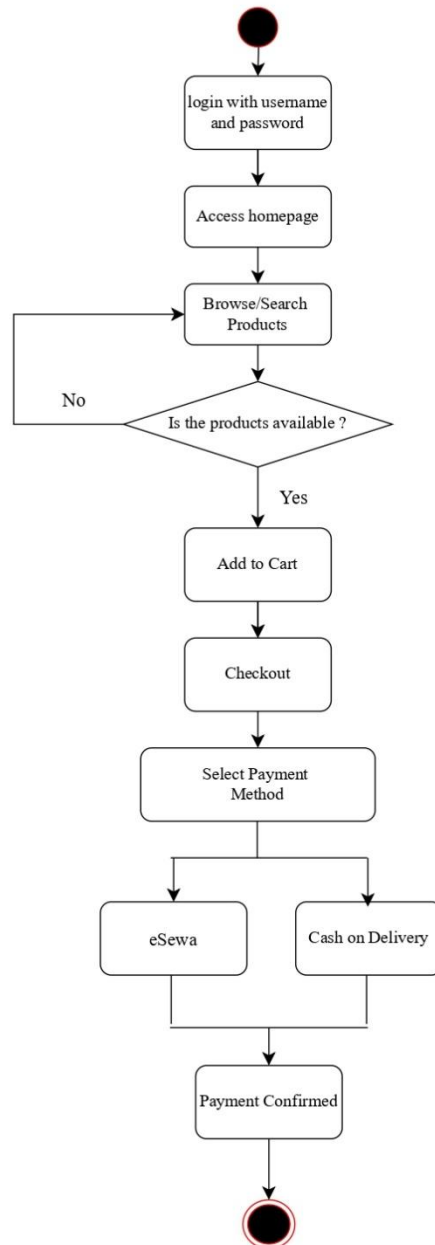


Figure 2.5: Activity Diagram

2.3.2 Sequence Diagram

This diagram illustrates the operational flow of the Luniva E-commerce System, focusing on the time-ordered interactions between the users, the application logic, and the database services. It is divided into two primary flows that define the system's behavior:

- i. **User Flow (Purchase Process):** This captures the complete customer journey from the initial authentication and product searching to the final transaction. It details how JavaScript handles cart updates on the client side, how the Django backend verifies product availability in the SQLite database, and the secure handshake with the eSewa API for payment processing. The flow concludes

with a secure order confirmation stored in the database and displayed to the user.

- ii. **Admin Flow (Management Process):** This details the administrative side of the system, focusing on inventory control and order oversight. It shows the interaction between the Admin and the Django Admin Panel, including the CRUD operations (Add/Update/Delete products) that modify the SQLite tables. It also illustrates how the Admin monitors incoming orders and updates payment or delivery statuses based on verified data.

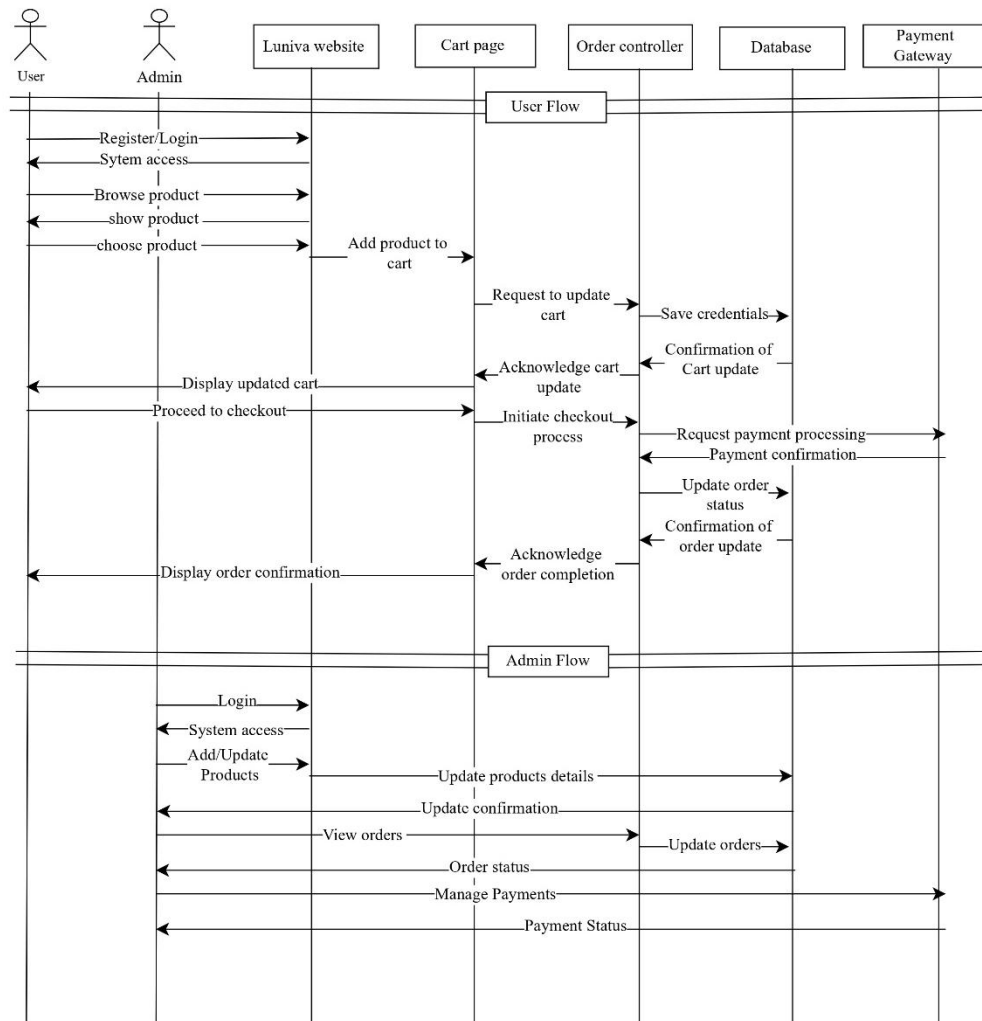


Figure 2.6: Sequence diagram

2.3.3 Entity-Relationship Diagram (ERD)

An Entity-Relationship Diagram (ERD) is a visual representation of the database structure, showing the entities, their attributes, and the relationships between them. For the Luniva E-commerce System, the ERD defines how data is structured, organized, and retrieved within the SQLite environment.

Key Components of the ERD:

A. Entities: These are the primary data structures for which data is stored in the database, represented as rectangles in the diagram:

- **User:** This entity stores the details of registered customers and admin profiles.
- **Product:** It stores comprehensive information regarding the items for sale, including pricing and stock status.
- **Category:** It represents the different groups or classifications of products.
- **Order:** It represents the final purchase transaction made by the customer.
- **OrderItem:** A junction table representing products within each order (handles multiple products per order).

Table 2.2: User Entity

Field Name	Data Type	Key Type	Description
user_id	Integer	PK	Unique identifier for each user
username	String	-	Unique username for login
email	String	-	Email address for notifications
phone	String	-	Contact number of the user
address	String	-	Address of the user
date_joined	Date	-	Account creation date
role	String	-	User role (customer/admin)

Table 2.3: Product Entity

Field Name	Data Type	Key Type	Description
product_id	Integer	PK	Unique identifier for each product
product_name	String	-	Name of the product
slug	String	-	URL-friendly version of name
price	Float	-	Selling price of the product
stock_status	Boolean	-	Product availability
category_id	Integer	FK	References Category entity

Table 2.4: Category Entity

Field Name	Data Type	Key Type	Description
category_id	Integer	PK	Unique identifier of category
category_name	String	-	Name of the category
created_at	Date	-	Date when category was added

Table 2.5: Order Entity

Field Name	Data Type	Key Type	Description
order_id	Integer	PK	Unique identifier for each order
user_id	Integer	FK	References User entity
order_date	Date	-	Date when order was placed
delivery_date	Date	-	Scheduled delivery date
total_price	Float	-	Total cost of the order
payment_status	String	-	Status of payment (Paid/Unpaid)

Table 2.6: OrderItem Entity

Field Name	Data Type	Key Type	Description
order_item_id	Integer	PK	Unique identifier for order item
order_id	Integer	FK	References Order entity
product_id	Integer	FK	References Product entity
quantity	Integer	-	Quantity of the product ordered
price	Float	-	Price of product at order time

B. Attributes: Attributes are the specific data fields for each entity. Based on the system schema:

- **Users:** user_id (PK), username, email, phone, date_joined.
- **Products:** product_id (PK), product_name, slug, price, stock_status, category_id (FK).
- **Category:** category_id (PK), category_name, created_at.
- **Orders:** order_id (PK), user_id (FK), product_id (FK), order_date, delivery_date, total_price, payment_status.
- **Admin:** admin_id (PK), admin_name, email, role, date_created.

C. Relationships:

- User → Order (1:M): A user can place multiple orders.
- Category → Product (1:M): Each category can include multiple products.
- Order → OrderItem (1:M): Each order can contain multiple products.
- Product → OrderItem (1:M): A product can appear in multiple order items.

D. Keys:

Primary Key (PK): Unique identifiers for each table (user_id, product_id, category_id, order_id, order_item_id)

Foreign Key (FK): Links between tables to enforce relationships:

- Product.category_id → Category.category_id
- Order.user_id → User.user_id
- OrderItem.order_id → Order.order_id
- OrderItem.product_id → Product.product_id

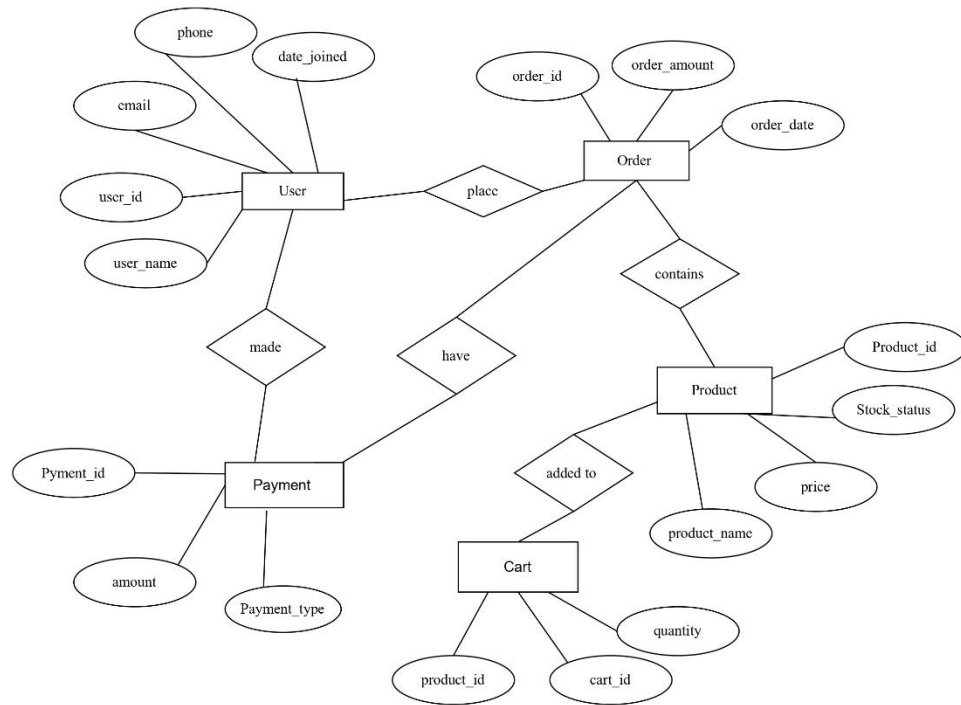


Figure 2.7: ER Diagram

2.4 System Implementation

The implementation phase involves the actual coding and integration of the system components. It translates the design models into a functioning application using the selected technology stack.

2.4.1 Tools and Technologies Used

The system is built using a robust stack featuring a Django backend and Django Templates enhanced with JavaScript for the frontend, ensuring high performance, security, and a smooth e-commerce experience.

- **Frontend (Django Templates & JS):** Builds a dynamic and responsive user interface using HTML, CSS, and Bootstrap. JavaScript is used to handle real-time shopping cart updates and form validations without requiring page refreshes.
- **Backend (Django):** A Python-based web framework that manages user authentication, product logic, and communication between the frontend and the database.
- **Database (SQLite):** A lightweight, serverless relational database used to store structured data such as user profiles, product inventory (categories and items), and order records.

- **Authentication:** Uses Django's built-in authentication for secure login and session management, providing separate access levels for customers and administrators.
- **Payment Gateway (eSewa API):** Provides secure online payment integration with HMAC-SHA256 signature verification to ensure transaction security during the checkout process.
- **Visual Studio Code (VS Code):** The primary Integrated Development Environment (IDE) used for writing Python, JavaScript, and HTML/CSS code.
- **Bootstrap:** Utilized for creating a mobile-responsive and clean UI design to ensure accessibility across various devices.
- **Draw.io:** A web-based tool used for modeling the system architecture, Activity Diagrams, and ERDs included in this report.
- **Git & GitHub:** Used for version control and source code management to ensure systematic development and backup of the Luniva codebase.

2.4.2 Module Description

The Luniva E-commerce System is developed using a modular approach, ensuring each functional area is independent and maintainable. The following modules represent the core logic of the application:

i. User Authentication & Profile Module This module manages the entry points and identity of the users. It handles registration and login to ensure secure access to the e-commerce platform. Once logged in, users can manage their personal profiles and track their previous orders.

- **Registration:** New users can create an account with a unique username and encrypted password.
- **Login:** Existing users access the system using their credentials, managed by Django's session framework.
- **Profile:** Displays user-specific information such as Full Name, Email, and Order History.

ii. Product & Category Management Module This module allows users to browse items and administrators to manage the store's inventory.

- **Browsing:** Customers can filter products based on categories and view details via Slug-based URLs.
- **Search:** A functional search bar allows users to query the SQLite database for specific products.

- **Admin Control:** Administrators can add, update, or delete products and categories through a secure dashboard.

iii. Shopping Cart & Order Module: This is the core functional module where JavaScript interacts with the Django backend.

- **Cart Logic:** Users can add items to their cart, update quantities, and remove items. JavaScript ensures these changes are reflected instantly.
- **Checkout:** Gathers shipping information and calculates the total price from the database.
- **Payment:** Integrates the eSewa payment flow, allowing for digital verification or selecting Cash on Delivery (CoD).

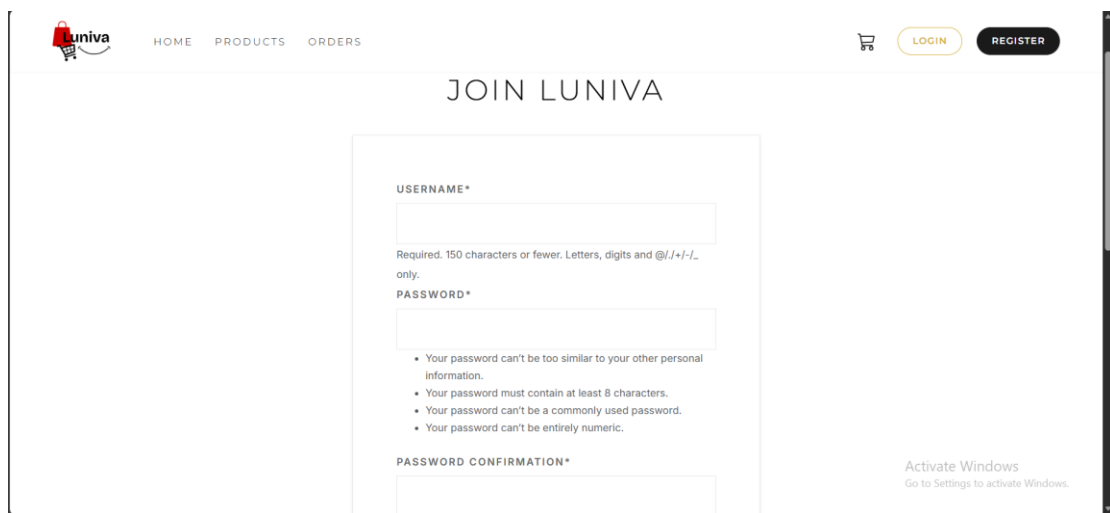


Figure 2.8: Registration Form

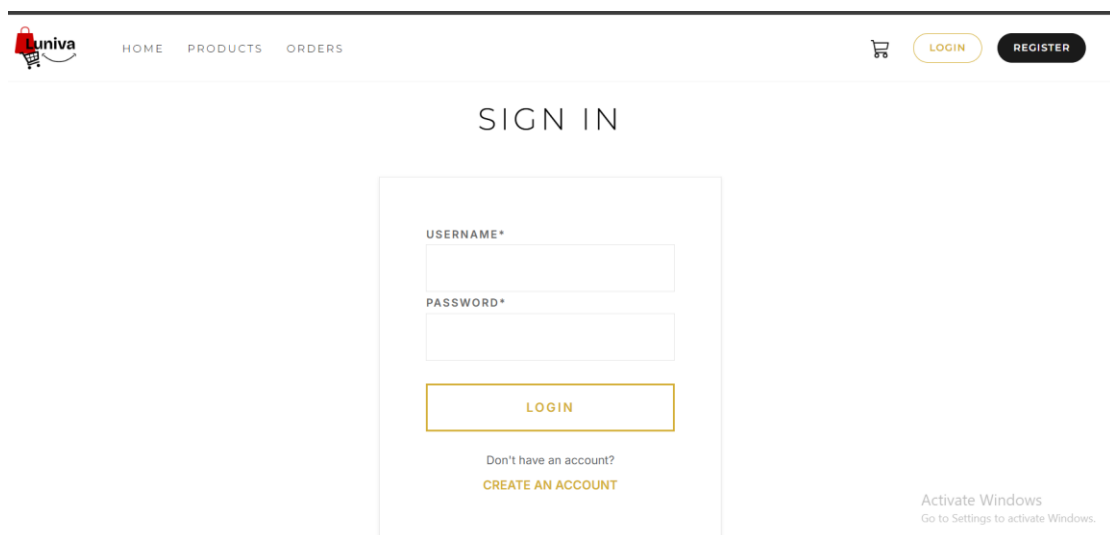


Figure 2.9: Login Form

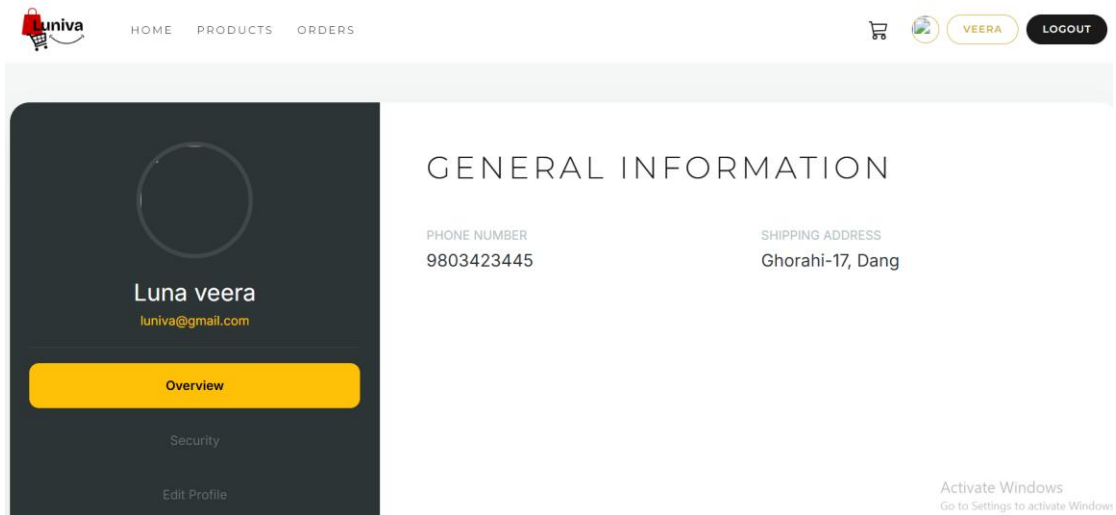


Figure 2.10: User Profile

Product & Category Module: This module is responsible for the discovery phase of the application. It dynamically fetches a list of available products from the SQLite database and displays them based on their respective classifications, such as Electronics, Clothing, or Groceries.

- **Product Listing:** Shows product names (e.g., Gaming Laptop, Cotton T-shirt) along with their current selling prices and professional images.
- **Search Logic:** Allows users to search for specific items using the search bar in the navigation header, which queries the database for matching product titles or descriptions.
- **Slug-based Navigation:** Uses URL-friendly "slugs" to provide clean, readable links for each product, improving the overall user experience.
- **Stock Status:** Tracks the stock_status of each item to display whether a product is "In Stock" or "Out of Stock" before a user attempts to add it to the cart.

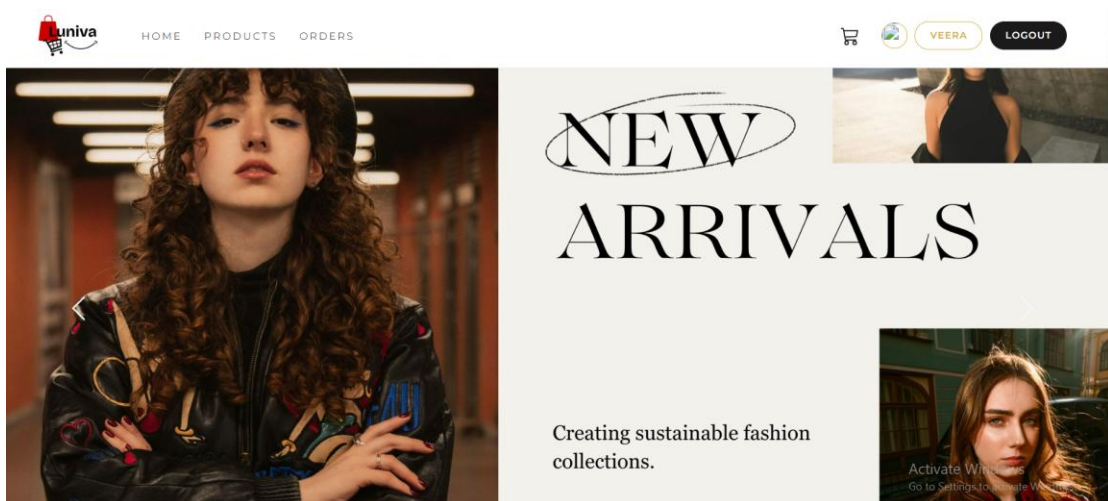


Figure 2.11: Home Page

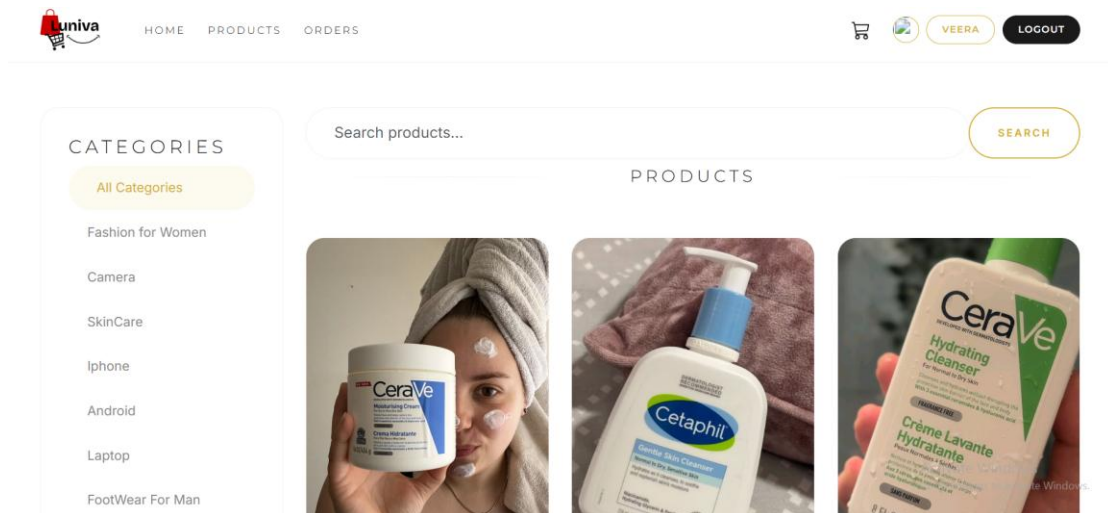


Figure 2.12: Product Listing

Shopping Cart & Order Module: This module serves as the core of the transaction process, utilizing JavaScript to handle real-time user interactions and the Django backend to finalize purchases.

- **Asynchronous Cart (JS):** Enables users to add products to their cart, change quantities, or remove items without refreshing the page.
- **Order Initialization:** Gathers the selected items and calculates the total price, including any applicable shipping fees, before proceeding to checkout.
- **Payment Integration:** Connects the system to the eSewa API for digital payment verification or provides a manual Cash on Delivery (CoD) option.
- **Order Tracking:** Records the transaction in the Orders DB and allows users to view the current status of their purchase (e.g., Pending, Processing, or Delivered).

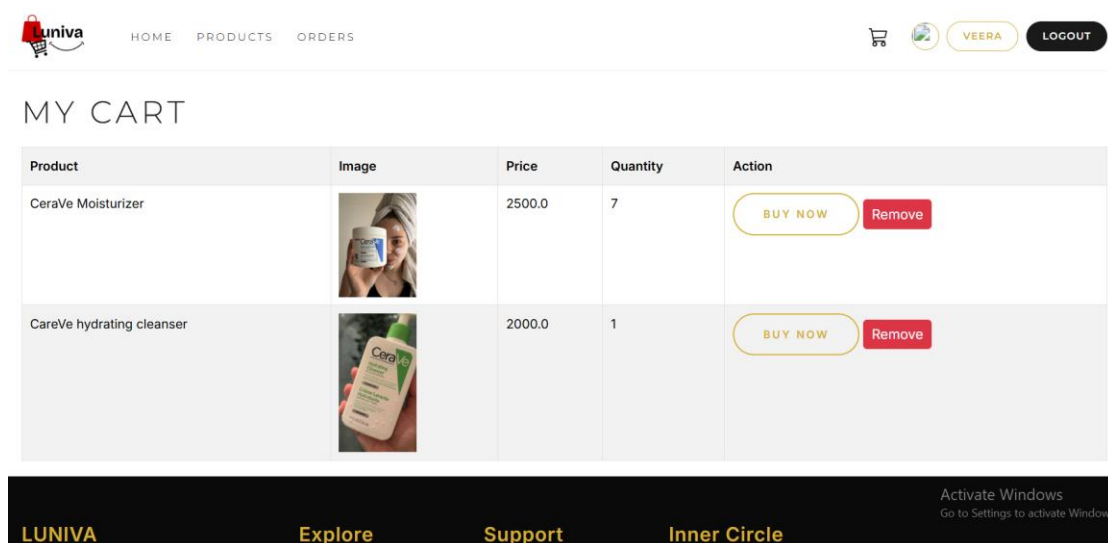


Figure 2.13: My Cart

SECURE CHECKOUT

QUANTITY*

ADDRESS*

CONTACT NUMBER*

PAYMENT METHOD*

PLACE ORDER

Activate Windows
Go to Settings to activate Windows

Figure 2.14: Checkout Form

Payment & Transaction Module: This module handles the financial transactions of the system. It ensures that every order is linked to a valid payment record in the SQLite database, providing a transparent trail for both customers and administrators.

- **Digital Payment:** Integrates the eSewa API for secure digital transactions, managing the generation of unique transaction IDs, signature verification, and real-time payment confirmation.
- **Manual Payment:** Supports a Cash on Delivery (CoD) option, allowing users to bypass the digital gateway while still recording the order in the database with a "Pending" payment status until the item is delivered.
- **Security Verification:** Implements backend checks to ensure that the payment amount matches the order total, protecting the system against price manipulation during the checkout process.

1

ADDRESS*

CONTACT NUMBER*

PAYMENT METHOD*

Cash On Delivery

Esewa

GUARANTEED SECURE TRANSACTION

Figure 2.15: Choose Payment Method

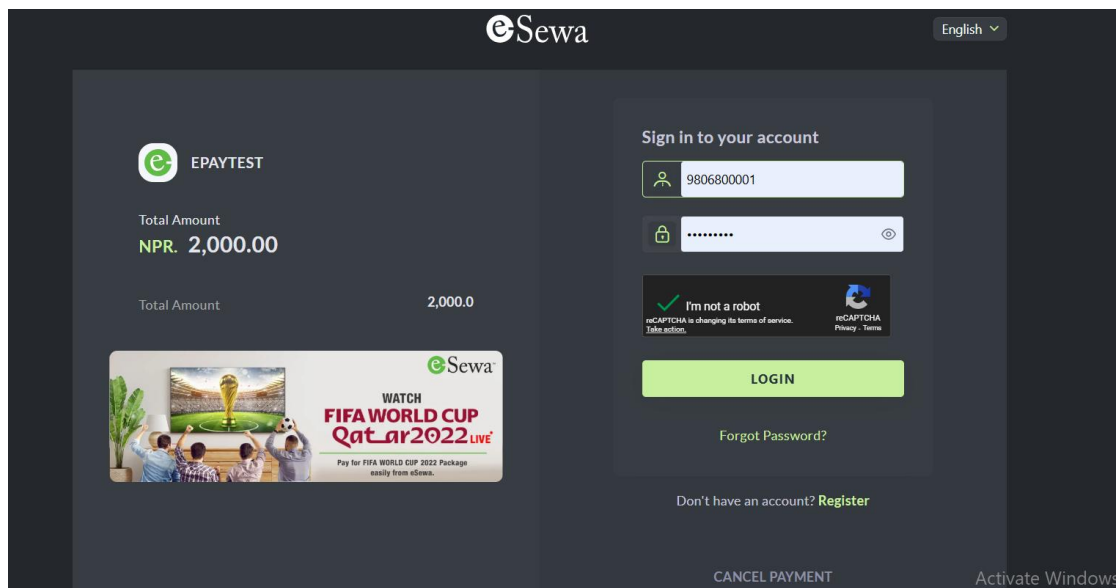


Figure 2.16: Esewa Login Page

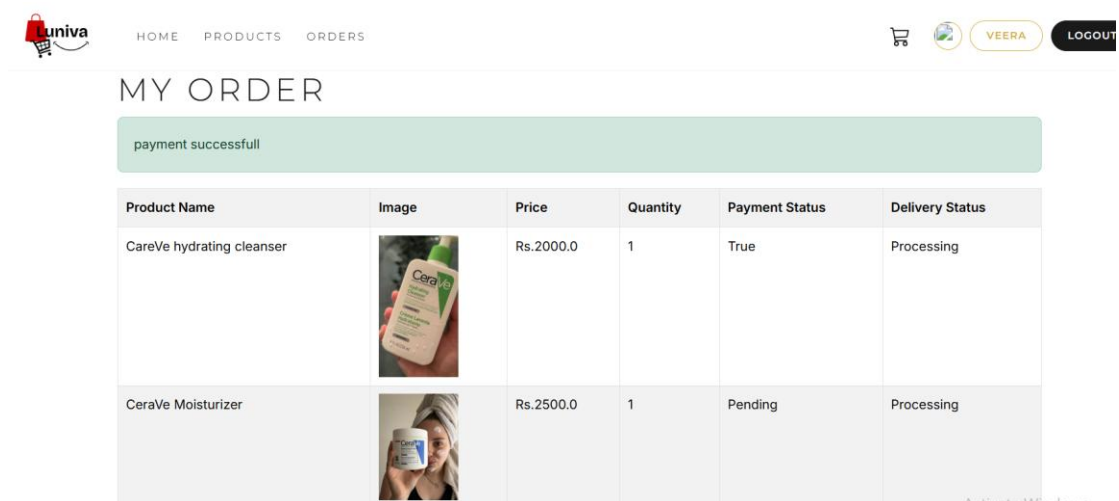


Figure 2.17: Successful Payment

Admin Management Module: The Admin Module is the "Control Center" of the Luniva E-commerce System. While regular users only see the storefront and their shopping cart, the Administrator uses this module to manage the entire business, from the product inventory to the customer orders and payment statuses.

Admin Dashboard (The Overview): When the admin logs in, they see a summary of everything happening in the system. It shows a birds-eye view of the total number of products, registered users, and recent sales activities. This dashboard allows the admin to monitor the health of the e-commerce platform at a glance.

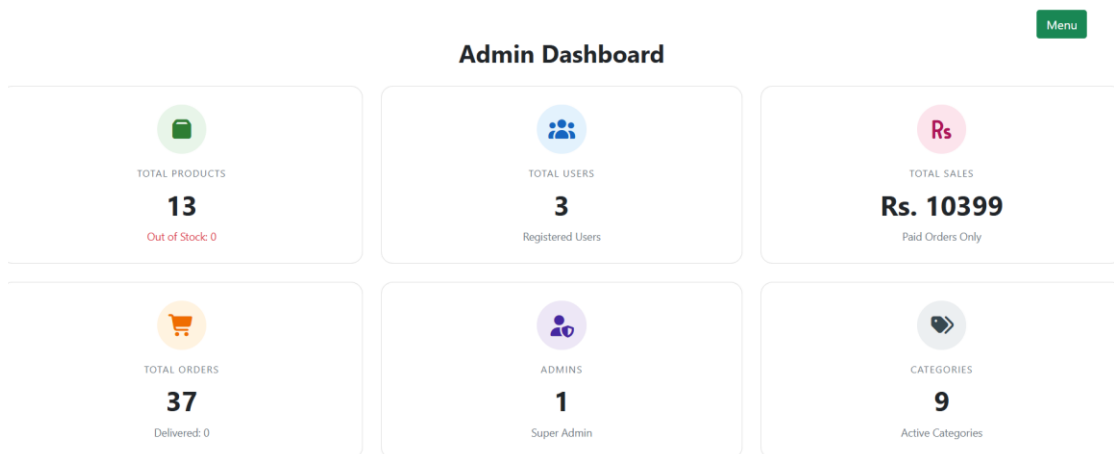


Figure 2.18: Admin Dashboard

Inventory & Category Management: The admin can keep the website up to date by adding new products or changing categories. If a new item is added to the store, the admin uses a form to upload the product name, price, stock status, and photo. This ensures that the frontend always displays current and available stock.

Add Product

Product name*

Product price*

Stock quantity*

Product image

No file chosen

Product description*

Category*

Slug*

Figure 2.19: Add Product

User & Order Management: This part allows the admin to see every registered customer and every purchase made on the site. They can check the Order Date,

Payment Status (verified via eSewa or CoD), and total price for every transaction to make sure the fulfillment process is running smoothly.

Database Control (SQLite): Since the system uses SQLite, the admin module provides an interface to directly manage the relational data. The administrator can update category names or delete outdated product records, ensuring that the database remains clean and optimized.

2.4.3 Testing

System testing is the process of evaluating the application to ensure it meets the specified requirements and is free of critical bugs. For Luniva, testing was conducted across several layers, ensuring that the Django backend, JavaScript cart, and eSewa integration work harmoniously.

Testing was performed on a module-by-module basis:

- **Input validation** for registration, login, and checkout forms.
- **Database checks** for accurate storage of users, product inventory, and order records in SQLite.
- **Session verification** for secure access and persistence of the shopping cart.
- **Functional testing** of admin and user roles to ensure proper permission levels.
- **UI testing** for smooth navigation and responsiveness across different screen sizes.

Table 2.7: Test Case 1

Test Case ID	Module	Test Scenario	Test Steps	Expected Result	Status
TC001	User	Verify user registration	Enter valid data in registration form and submit	User should be registered and redirected to login	Pass

Table 2.8: Test Case 2

Test Case ID	Module	Test Scenario	Test Steps	Expected Result	Status
TC002	User Login	Verify login functionality	Enter valid username and password	User should be logged in and taken to homepage	Pass

Table 2.9: Test Case 3

Test Case ID	Module	Test Scenario	Test Steps	Expected Result	Status
TC003	Product Order	Submit a product order	Add items to cart and click checkout	Order should be saved and status set to "Pending"	Pass

Table 2.10: Test Case 4

Test Case ID	Module	Test Scenario	Test Steps	Expected Result	Status
TC004	Admin Order	Manage product order	Admin selects order and updates status (shipped/delivered)	Order status should be updated correctly in the system	Pass

Table 2.11: Test Case 5

Test Case ID	Module	Test Scenario	Test Steps	Expected Result	Status
TC005	Payment Processing	Verify payment functionality	User completes checkout and submits via eSewa	Payment processed successfully and confirmation displayed	Pass

Table 2.12: Test Case 6

Test Case ID	Module	Test Scenario	Test Steps	Expected Result	Status
TC006	User Order History	View past orders	User logs in and navigates to order history page	All past orders should be displayed correctly	Pass

Table 2.13: Test Case 7

Test Case ID	Module	Test Scenario	Test Steps	Expected Result	Status
TC007	User Logout	Verify logout functionality	User clicks the logout button	User session terminated and redirected to the login page	Pass

Table 2.14: Test Case 8

Test Case ID	Module	Test Scenario	Test Steps	Expected Result	Status
TC008	Product Search	Verify product search	User enters product name in search bar	System should display items matching the search criteria	Pass

Chapter 3: Conclusion and Recommendation

3.1 Summary

This project report presents the design and development of Luniva, an e-commerce system developed to simplify online shopping and inventory management for local retail businesses. The project was initiated to address common challenges faced by small and medium-sized enterprises, such as manual sales tracking, unorganized stock management, and limited customer reach beyond physical store locations.

The system was developed using Django Templates (HTML, CSS, Bootstrap) and JavaScript for the frontend, Django (Python) for backend processing, and SQLite as the database. These technologies were selected to create a structured, cost-effective, and user-friendly web application suitable for small-scale deployment.

An Agile development methodology was followed, allowing the system to be built incrementally through iterative development and continuous testing. This approach supported flexible requirement handling and frequent refinement of features such as user authentication, product management, shopping cart operations, and payment processing. The system enables customers to register, browse products using search and category filters, manage a real-time shopping cart, and place orders. Administrators are provided with a centralized dashboard to manage inventory, monitor orders, and verify payments.

Key Achievements

- **Systematic Planning:** Core e-commerce workflows were analyzed to address issues such as stock inconsistency and inefficient order handling.
- **Modular Development:** Separate modules were implemented for user authentication, product and category management, shopping cart functionality, and order processing.
- **Payment Integration:** The system supports digital payment through the eSewa gateway as well as a Cash on Delivery (CoD) option, increasing flexibility for users.
- **Testing:** Functional testing was conducted across multiple scenarios to verify system reliability, correct data storage in SQLite, and responsive behavior across devices.

Overall, the system provides a structured digital solution that helps local retailers transition from manual or social media-based selling to an organized e-commerce platform.

3.2 Conclusion

The Luniva E-commerce System successfully fulfills the objectives defined at the beginning of the project. Many local retailers in regions such as Dang continue to rely on traditional selling methods or informal social media platforms, which lack proper order tracking, inventory control, and secure payment verification. This project addresses those limitations by offering a dedicated and automated e-commerce platform.

Based on the development and testing outcomes, the following conclusions are drawn:

- **Achievement of Project Objectives:** The system effectively automates key retail operations, reducing reliance on manual record-keeping and minimizing order management errors.
- **Digital Enablement of Local Retail:** Luniva provides a structured web-based storefront that replaces informal selling practices with an organized system.
- **Improved User Experience:** Customers can search products, view availability status, and manage shopping carts dynamically using JavaScript-based interactions.
- **Efficient Administrative Control:** Administrators can manage products, update inventory, and track order status through a secure dashboard.
- **Secure Payment Handling:** Integration with the eSewa payment gateway ensures that digital transactions are verified before order confirmation.
- **Learning Outcome:** The project enhanced practical understanding of full-stack web development using Django, relational database handling with SQLite, and asynchronous frontend interactions using JavaScript.

The project demonstrates that a well-structured, small-scale e-commerce system can significantly improve operational efficiency for local businesses.

3.3 Recommendation

Based on observations during development and testing, several enhancements are recommended to improve the functionality and scalability of Luniva in the future:

- **Email and SMS Notifications:** Automated notifications for order confirmation and delivery updates would enhance communication and customer trust.
- **Mobile Application Development:** A dedicated Android or iOS application could improve accessibility and user engagement through push notifications and optimized interfaces.

- **Analytics Dashboard:** Incorporating basic data analytics for administrators, such as sales trends and popular products, would support informed decision-making.
- **Invoice Generation:** Automatic generation of downloadable PDF invoices after successful payments would improve transaction transparency.
- **Multi-Vendor Support:** Extending the system to support multiple sellers would allow Luniva to evolve into a marketplace platform.
- **Recommendation System:** Implementing product suggestions based on browsing or purchase history could enhance the user experience.
- **Database Migration:** For higher traffic and larger datasets, migrating from SQLite to PostgreSQL or MySQL is recommended to improve performance and scalability.

References

Bootstrap Team. (2025). Bootstrap documentation.

<https://getbootstrap.com/docs>

Django Software Foundation. (2025). Django documentation (Version 4.2).

<https://docs.djangoproject.com/en/4.2/>

Draw.io. (2025). Diagrams.net: Online diagramming tool. <https://www.diagrams.net/>

eSewa. (2025). eSewa API integration guide. <https://esewa.com.np/developers>

MDN Web Docs. (2025a). CSS: Cascading Style Sheets. Mozilla.

<https://developer.mozilla.org/en-US/docs/Web/CSS>

MDN Web Docs. (2025b). HTML: HyperText Markup Language. Mozilla.

<https://developer.mozilla.org/en-US/docs/Web/HTML>

MDN Web Docs. (2025c). JavaScript reference. Mozilla.

<https://developer.mozilla.org/en-US/docs/Web/JavaScript>

Python Software Foundation. (2025). Python 3.12 documentation.

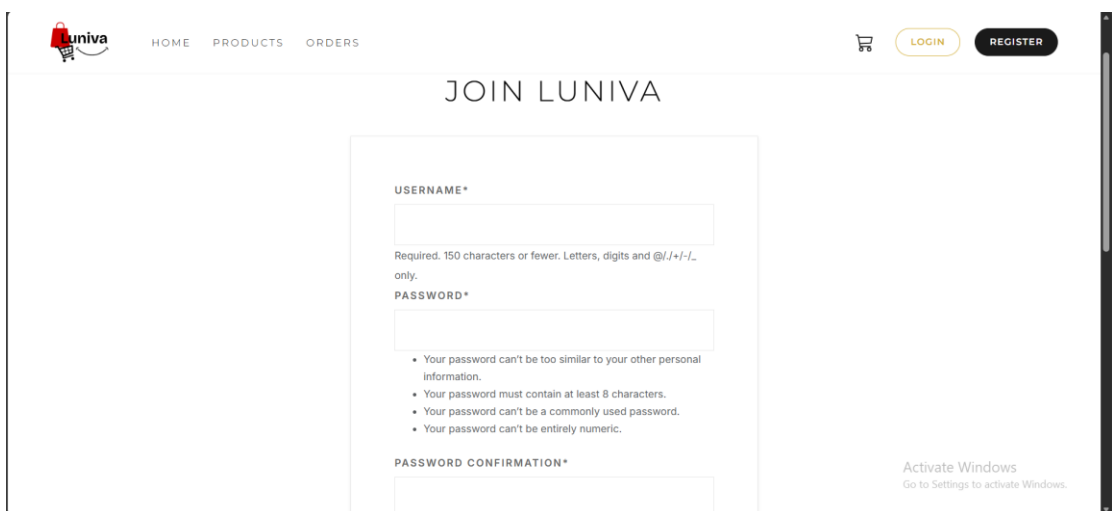
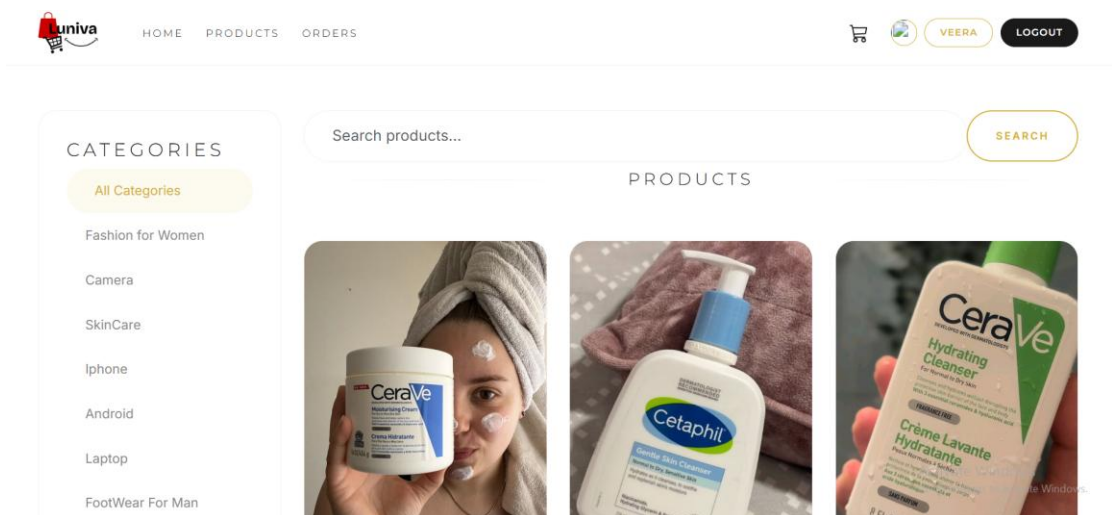
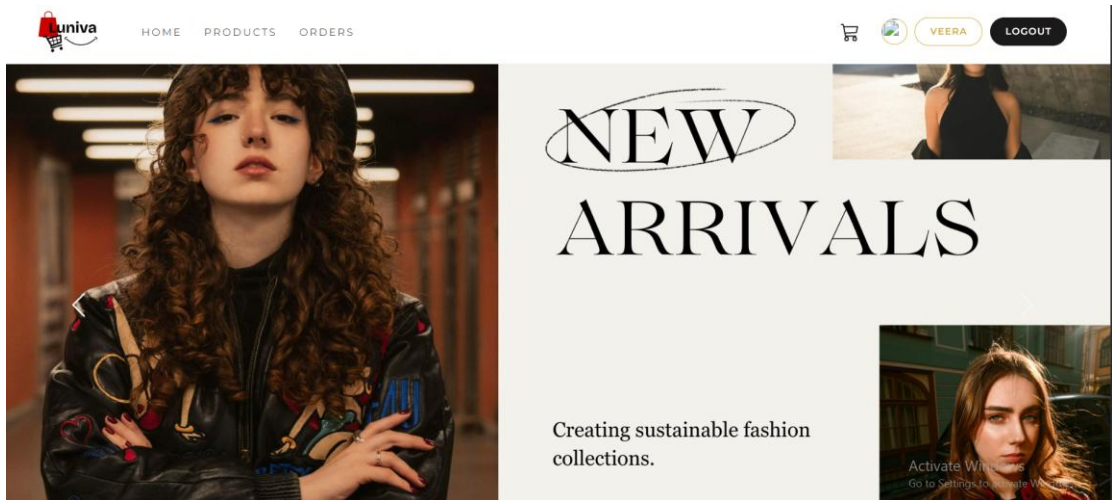
<https://docs.python.org/3/>

Sommerville, I. (2011). Software engineering (9th ed.). Addison-Wesley.

SQLite. (2025). SQLite documentation.

<https://www.sqlite.org/docs.html>

Appendices



SIGN IN

USERNAME*

PASSWORD*

LOGIN

Don't have an account?

[CREATE AN ACCOUNT](#)

Activate Windows

Go to Settings to activate Windows.

Luna veera

[luniva@gmail.com](#)

Overview

Security

Edit Profile

GENERAL INFORMATION

PHONE NUMBER

9803423445

SHIPPING ADDRESS

Ghorahi-17, Dang

Activate Windows

Go to Settings to activate Windows.

MY CART

Product	Image	Price	Quantity	Action
CeraVe Moisturizer		2500.0	7	BUY NOW Remove
CareVe hydrating cleanser		2000.0	1	BUY NOW Remove

LUNIVA

Explore

Support

Inner Circle

Activate Windows

Go to Settings to activate Windows.


[HOME](#)
[PRODUCTS](#)
[ORDERS](#)



VEERA
LOGOUT

SECURE CHECKOUT

QUANTITY*

ADDRESS*

CONTACT NUMBER*

PAYMENT METHOD*

PLACE ORDER

Activate Windows
Go to Settings to activate Windows


[HOME](#)
[PRODUCTS](#)
[ORDERS](#)



VEERA
LOGOUT

1

ADDRESS*

CONTACT NUMBER*

PAYMENT METHOD*

Cash On Delivery

Esewa

GUARANTEED SECURE TRANSACTION


English



EPAYTEST

Total Amount

NPR. 2,000.00

Total Amount

2,000.0



Sign in to your account








LOGIN

Forgot Password?

Don't have an account? Register

CANCEL PAYMENT

Activate Windows

34



MY ORDER

payment successfull

Product Name	Image	Price	Quantity	Payment Status	Delivery Status
CareVe hydrating cleanser		Rs.2000.0	1	True	Processing
CeraVe Moisturizer		Rs.2500.0	1	Pending	Processing

Artista Window

Admin Dashboard

[Menu](#)

TOTAL PRODUCTS

13

Out of Stock: 0



TOTAL USERS

3

Registered Users



TOTAL SALES

Rs. 10399

Paid Orders Only



TOTAL ORDERS

37

Delivered: 0



ADMINS

1

Super Admin



CATEGORIES

9

Active Categories